

- Vectors in underline, matrices in capital. Assume all matrices have real entries unless stated otherwise.
 - For a vector $\underline{x}$, we denote by x_i the i^{th} entry of $\underline{x}$.
 - The inequality $A \geq 0$ means that the matrix A is positive semi definite. The inequality $\underline{x} \geq 0$ means that all the entries of the vector $\underline{x}$ are non negative.
 - Most of these problems are adapted from [1, 2] and related resources on the web.
 - You may use either of `CVXPY` (Python), `CVX.jl` (Julia) or `CVX` (matlab) to solve these problems.
1. *Regression with Corrupted Measurements:* You are given an m -size dataset with n features in a data matrix $A \in \mathbb{R}^{m \times n}$ and a measurement vector $\underline{y}$. However, the observed vector $\underline{y}$ contains corruptions. You do not know *a priori* which measurements are corrupted.
 - (a) *The Prescient Solution:* If you *did* know the indices of the corrupted measurements in $\underline{y}$, explain how you would find the optimal $\underline{x}$. Call this the *prescient* solution.
 - (b) *The Robust Solution:* If you did not know which measurements are corrupted, you may formulate the problem as a penalty function minimization as discussed in class:

$$\min_{\underline{x}} \sum_{i=1}^m \phi(r_i) \quad \text{where} \quad r_i = (A\underline{x})_i - y_i$$

Simulation: For the purpose of testing your procedure, follow these steps to generate the data:

- Pick $m = 450$ measurements, $n = 300$ regressors.
- Generate the ground truth vector $\underline{x}^{\text{true}} \sim \mathcal{N}(0, I)$ (i.e., each entry is i.i.d Gaussian).
- Generate the entries of the data matrix A similarly (each entry is i.i.d Gaussian with zero mean and unit variance).
- Set $\underline{y}_{\text{orig}} = A\underline{x}^{\text{true}} + \underline{\epsilon}$, where $\underline{\epsilon} \sim \mathcal{N}(0, I)$.
- Create the observed $\underline{y}$ by flipping the sign of each entry in $\underline{y}_{\text{orig}}$ with probability p , independently for each entry.

Use this resulting $\underline{y}$ and the matrix A as inputs to the algorithm for part (b). Solve using the various penalty functions discussed in class (e.g., ℓ_2 , ℓ_1 , ℓ_∞ , Huber, etc).

For each choice of penalty function, obtain the estimate $\hat{\underline{x}}$ and compare it with $\underline{x}^{\text{true}}$. You may take the average over multiple problem instances (e.g., 50 trials) to compute the normalized error:

$$\mathbb{E} \left[\frac{\|\hat{\underline{x}} - \underline{x}^{\text{true}}\|_2}{\|\underline{x}^{\text{true}}\|_2} \right]$$

Plot the above error as a function of p (e.g., for $p \in [0, 0.5]$), for each of the penalty functions. Compare with the error for the prescient solution as well. Which of the penalties gives a solution closest to the ground truth $\underline{x}^{\text{true}}$?

Remark: Note that the generation model used in the simulations is linear with Gaussian noise. As we will see later, this predisposes you towards ℓ_2 -norm minimization. However, the presence of incorrect measurements makes this solution suboptimal.

2. *Path planning with contingencies.* A vehicle path down a (straight, for simplicity) road is specified by a vector $p \in \mathbf{R}^N$, where p_i gives the position perpendicular to the centerline at the point ih meters down the road, where $h > 0$ is a given discretization size. (Throughout this problem, indexes on N -vectors will

correspond to positions on the road.) We normalize p so $-1 \leq p_i \leq 1$ gives the road boundaries. (We are modeling the vehicle as a point, by adjusting for its width.) You are given the initial two positions $p_1 = a$ and $p_2 = b$ (which give the initial road position and angle), as well as the final two positions $p_{N-1} = c$ and $p_N = d$.

You know there may be an obstruction at position $i = O$. This will require the path to either go around the obstruction on the left, which requires $p_O \geq 0.5$, or on the right, which requires $p_O \leq -0.5$, or possibly the obstruction will clear, and the obstruction does not place any additional constraint on the path. These are the three *contingencies* in the problem title, which we label as $k = 1, 2, 3$.

You will plan three paths for these contingencies, $p^{(k)} \in \mathbf{R}^N$ for $k = 1, 2, 3$. They must each satisfy the given initial and final two road positions and the constraint of staying within the road boundaries. Paths $p^{(1)}$ and $p^{(2)}$ must satisfy the (different) obstacle avoidance constraints given above. Path $p^{(3)}$ does not need to satisfy an avoidance constraint.

Now we add a twist: You will not learn which of the three contingencies will occur until the vehicle arrives at position $i = S$, when the sensors will determine which contingency holds. We model this with the *information constraints* (also called *causality constraints* or *non-anticipatory constraints*),

$$p_i^{(1)} = p_i^{(2)} = p_i^{(3)}, \quad i = 1, \dots, S,$$

which state that before you know which contingency holds, the three paths must be the same.

The objective to be minimized is

$$\sum_{k=1}^3 \sum_{i=2}^{N-1} (p_{i-1}^{(k)} - 2p_i^{(k)} + p_{i+1}^{(k)})^2,$$

the sum of the squares of the second differences, which gives smooth paths.

- (a) Explain how to solve this problem using convex optimization.
- (b) Solve the problem instance defined by the following parameters:

```
# parameters
N = 120;
O = 60;
S = 30;
h = 0.5;
p1 = 0.3; p2 = 0.3;
pN_1 = 0.3; pN = 0.3;
```

Report the optimal objective value. Plot the optimal paths (on one plot). You may use the following code snippet to generate the plots (assumes the solution paths are stored in matrix P):

```
### FEEL FREE TO COPY THE FOLLOWING PLOTTING CODE ###
#figure(); ax = axes();
#x = collect(1 : N) * h;
#ax[:set_xlim]([x[1], x[N]]); ax[:set_ylim]([-1, 1]);
#plot([0 * h, 0 * h], [0.5, 1], "r--", label="left safe region");
#plot([0 * h, 0 * h], [-1, -0.5], "b--", label="right safe region");
#plot([S * h, S * h], [-1, 1], "k--", label="obstacle revealed");
#plot(x, P[:, 1], color="red", linewidth=2.0, label="obstacle left");
#plot(x, P[:, 2], color="blue", linewidth=2.0, label="obstacle right");
#plot(x, P[:, 3], color="green", linewidth=2.0, label="obstacle clear");
#legend(loc = 4, scatterpoints = 1);
#savefig("path_plan_contingencies.eps");
```

3. *Circularly symmetric Huber function.* The scalar Huber function (for $\mu = 1$) is defined as

$$f_{\text{hub}}(x) = \begin{cases} (1/2)x^2 & |x| \leq 1 \\ |x| - 1/2 & |x| > 1. \end{cases}$$

As seen in class, this came up in robust estimation. This problem concerns generalizations of the Huber function to $\mathbf{R}^n$. One generalization to $\mathbf{R}^n$ is given by $f_{\text{hub}}(x_1) + \cdots + f_{\text{hub}}(x_n)$ (this is the generalization you will use in problem 1(b)), but this function is not circularly symmetric, *i.e.*, invariant under transformation of x by an orthogonal matrix. A generalization to $\mathbf{R}^n$ that *is* circularly symmetric is

$$f_{\text{cshub}}(x) = f_{\text{hub}}(\|x\|_2) = \begin{cases} (1/2)\|x\|_2^2 & \|x\|_2 \leq 1 \\ \|x\|_2 - 1/2 & \|x\|_2 > 1. \end{cases}$$

(The subscript stands for ‘circularly symmetric Huber function’.) Show that f_{cshub} is convex.

4. *Geometric Means and SOCP.* Recall the following definition of Second Order Cone Programs (SOCP):

$$\begin{aligned} \min_x \quad & f^T x \\ \text{s.t.} \quad & \|A_i x + b_i\|_2 \leq c_i^T x + d_i, \quad i = 1, \dots, m, \\ & Fx = g \end{aligned}$$

- (a) *Hyperbolic Constraints:* Explain how a constraint of the type $xy \geq z^2$ with non-negative variables $x, y \geq 0$ can be written as an SOCP constraint.
 - (b) *Geometric Mean ($n = 2$):* Consider the problem of maximizing the geometric mean of x_1, x_2 subject to affine constraints on x_1, x_2 . Explain how this problem can be transformed into an SOCP.
 - (c) *Geometric Mean ($n = 2^k$):* Generalize your solution from (b) to the geometric mean of variables $x_1, x_2, \dots, x_m$, where m is a power of 2. Describe the construction of the constraints.
5. *Formulate the following approximation problems as LPs, QPs, SOCPs, or SDPs.* The problem data are $A \in \mathbf{R}^{m \times n}$ and $b \in \mathbf{R}^m$. The rows of A are denoted a_i^T .

- (a) *Deadzone-linear penalty approximation:* minimize $\sum_{i=1}^m \phi(a_i^T x - b_i)$, where

$$\phi(u) = \begin{cases} 0 & |u| \leq a \\ |u| - a & |u| > a, \end{cases}$$

where $a > 0$.

- (b) *Log-barrier penalty approximation:* minimize $\sum_{i=1}^m \phi(a_i^T x - b_i)$, where

$$\phi(u) = \begin{cases} -a^2 \log(1 - (u/a)^2) & |u| < a \\ \infty & |u| \geq a, \end{cases}$$

with $a > 0$.

- (c) *Huber penalty approximation:* minimize $\sum_{i=1}^m \phi(a_i^T x - b_i)$, where

$$\phi(u) = \begin{cases} u^2 & |u| \leq M \\ M(2|u| - M) & |u| > M, \end{cases}$$

with $M > 0$.

- (d) *Log-Chebyshev approximation:* minimize $\max_{i=1, \dots, m} |\log(a_i^T x) - \log b_i|$. We assume $b \succ 0$. An equivalent convex form is

$$\begin{aligned} \text{minimize} \quad & t \\ \text{subject to} \quad & 1/t \leq a_i^T x / b_i \leq t, \quad i = 1, \dots, m, \end{aligned}$$

with variables $x \in \mathbf{R}^n$ and $t \in \mathbf{R}$, and domain $\mathbf{R}^n \times \mathbf{R}_{++}$.

6. $\ell_{1.5}$ optimization. In this problem we study another natural method for blending the ℓ_1 and ℓ_2 norms, by using the $\ell_{1.5}$ -norm, defined as

$$\|z\|_{1.5} = \left(\sum_{i=1}^k |z_i|^{3/2} \right)^{2/3}$$

for $z \in \mathbf{R}^k$. We will consider the simplest approximation or regression problem:

$$\text{minimize} \quad \|Ax - b\|_{1.5},$$

with variable $x \in \mathbf{R}^n$, and problem data $A \in \mathbf{R}^{m \times n}$ and $b \in \mathbf{R}^m$. We will assume that $m > n$ and the A is full rank (*i.e.*, rank n). The hope is that this $\ell_{1.5}$ -optimal approximation problem should share some of the good features of ℓ_2 and ℓ_1 approximation.

- Give optimality conditions for this problem. Try to make these as simple as possible.
- (Optional:) Explain how to formulate the $\ell_{1.5}$ -norm approximation problem as an SDP. (Your SDP can include linear equality and inequality constraints.) *hint:* A constraint of the form $r^{3/2} \leq t$ can be written as

$$\begin{pmatrix} \sqrt{r} & r \\ r & t \end{pmatrix} \geq 0.$$

By introducing $0 \leq x \leq \sqrt{r}$, we may write the above as

$$\begin{pmatrix} x & r \\ r & t \end{pmatrix} \geq 0, \quad \begin{pmatrix} r & x \\ x & 1 \end{pmatrix} \geq 0$$

- Solve the specific numerical instance generated by the following code:

```
A=randn(100,30);
b=randn(100,1);
```

Numerically verify the optimality conditions. Give a histogram of the residuals, and repeat for the ℓ_2 -norm and ℓ_1 -norm approximations. You can use any method you like to solve the problem (but of course you must explain how you did it); in particular, you do not need to use the SDP formulation found in part (b), you may use, for e.g. `quad_over_lin(s,sqrt(s))`.

7. Find Moreau envelope and proximal points for

- $f: \mathbf{R} \mapsto \mathbf{R}$ given by $f(x) = \max\{x, 0\}$
- $f: \mathbf{R} \mapsto \mathbf{R}$ given by $f(x) = x^2$
- $f: \mathbf{R}^2 \mapsto \mathbf{R}$ given by $f(x, y) = \max\{|x|, |y|\}$

8. In class we discussed that the gradient of the Moreau envelope, defined by

$$f_\mu(\underline{x}) = \min_{\underline{y} \in \mathbf{R}^n} \left(f(\underline{y}) + \frac{1}{2\mu} \|\underline{x} - \underline{y}\|^2 \right), \quad \text{prox}_{f,\mu}(\underline{x}) = \underset{\underline{y} \in \mathbf{R}^n}{\text{argmin}} \left(f(\underline{y}) + \frac{1}{2\mu} \|\underline{x} - \underline{y}\|^2 \right).$$

In this exercise, we obtain the following expression for the gradient of f_μ

$$\nabla f_\mu(\underline{x}) = \frac{1}{\mu} (\underline{x} - \text{prox}_{f,\mu}(\underline{x})).$$

Recall that f_μ is convex, so if we obtain an expression of the form

$$f_\mu(\underline{x} + \underline{h}) \geq f_\mu(\underline{x}) + \underline{g}^\top \underline{h} \text{ for all } \underline{h},$$

then $\underline{g}$ must be the gradient of f_μ at $\underline{x}$.

- If $\underline{y}^* = \text{prox}_{f,\mu}(\underline{x})$, show that $\nabla f(\underline{y}^*) = (\underline{x} - \underline{y}^*)/\mu$.
- If f is convex, we must have

$$f(\underline{y}) \geq f(\underline{y}^*) + \nabla f(\underline{y}^*)^\top (\underline{y} - \underline{y}^*), \text{ for all } \underline{y}.$$

Use this in the defining expression for $f_\mu(\underline{x} + \underline{h})$ to obtain a lower bound on $f_\mu(\underline{x} + \underline{h})$.

- Derive the expression for ∇f_μ above.

References

- [1] Stephen Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.
- [2] Stephen Boyd and Lieven Vandenberghe. Additional exercises for convex optimization, 2010.